

Experiment – 4

Name: Renulasya

UID: 24BAI70210

Branch: CSE – AIML (CS221)

Section/Group: 24AIT-NTTP-2[A]

Semester: 5th

Date of Performance: 31/07/2026

Subject: Full Stack - II

Subject Code: 24CSP-337

EXPERIMENT – 4.1

1. **AIM:** To design and implement an interactive calendar interface for scheduling and managing posts.

2. **OBJECTIVE:**

1. To understand time-based data visualization in UI systems.
2. To implement a calendar-based scheduling interface.
3. To map structured post data to calendar events.
4. To enable user interaction through event scheduling and drag-and-drop.

3. **SOFTWARE REQUIREMENTS:**

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js

4. **THEORY**

An interactive calendar is a user interface component used to display and manage events based on date and time. In applications such as social media management systems, calendars help users schedule posts for future publication.

The **FullCalendar** library provides an attractive and responsive calendar with features such as month, week, and day views. Events can be displayed on specific dates and modified easily.

Each scheduled post is represented as an event and stored in the application state. Users can add new posts, click events to view details, and move events to different dates using drag-and-drop.

Interactive calendars improve usability, make scheduling easier, and provide a better user experience.

5. IMPLEMENTATION/PROCEDURE

1. Create a React application using create-react-app.
2. Install the FullCalendar packages.
3. Create a Calendar component.
4. Design the calendar user interface.
5. Create input fields for post title and date.
6. Store scheduled posts using React state.
7. Display events on the calendar.
8. Enable click events to view scheduled posts.
9. Enable drag-and-drop to reschedule events.
10. Execute the application using npm start.

6. CODE

Index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import "./App.css";
import App from "./App";

const root =
  ReactDOM.createRoot(document.getElementById("root"));

root.render(

  <React.StrictMode>

  <App />

  </React.StrictMode>

);
```

App.js

```
import "./App.css";
import CalendarComponent from
  "./components/CalendarComponent";

function App() {
  return (
    <div className="app">
      <h1> 📅 Social Media Post Scheduler</h1>
      <CalendarComponent />
    </div>
  );
}

export default App;
```

Calendarcomponnet.js

```
import { useState } from "react";
import FullCalendar from "@fullcalendar/react";
import dayGridPlugin from "@fullcalendar/daygrid";
import interactionPlugin from "@fullcalendar/interaction";

function CalendarComponent() {

  const [title, setTitle] = useState("");

  const [date, setDate] = useState("");

  const [events, setEvents] = useState([]);

  const handleAdd = () => {

    if(title===="" || date====""){

      alert("Enter title and date");

      return;

    }

    setEvents([

      ...events,

      {

        title:title,

        date:date

      }

    ]);

    setTitle("");

    setDate("");

  };

  return(

    <div>

      <div className="form">

        <input

          type="text"

          placeholder="Enter Post Title"

          value={title}

          onChange={(e)=>setTitle(e.target.value)}

        />
```

```
<input
  type="date"
  value={date}
  onChange={(e)=>setDate(e.target.value)}
/>

<button onClick={handleAdd}>

Schedule Post

</button>

</div>

<FullCalendar
  plugins={[dayGridPlugin,interactionPlugin]}
  initialView="dayGridMonth"
  editable={true}
  selectable={true}
  events={events}
  eventClick={(info)=>{
    alert("Post : "+info.event.title);
  }}
/>

</div>

);
}

export default CalendarComponent;
App.css
body{
  margin:0;

  font-family:Arial;

  background:linear-gradient(to right,#4facfe,#00f2fe);
}

.app{
  width:90%;
```



```
margin:auto;

padding:20px;

}

h1 {

text-align:center;

color:black;
font-size:45px;
margin-bottom:30px;
text-shadow:2px 2px 8px rgba(0,0,0,0.3);
}

.form {

display:flex;

justify-content:center;

gap:10px;

margin-bottom:20px;

}

input {

padding:12px;
border-radius:10px;
border:2px solid #ddd;
font-size:16px;
width:220px;
}

button {
background:linear-gradient(45deg,#7b2ff7,#9d4edd);
color:white;
border:none;
padding:12px 20px;
border-radius:10px;
cursor:pointer;
font-size:15px;
font-weight:bold;
transition:0.3s;
}

button:hover {
transform:scale(1.05);
}

.fc {
width:100%;
max-width:1200px;
margin:20px auto;
background:white;
padding:20px;
border-radius:20px;
box-shadow:0 8 25px rgba(0,0,0,0.2);
}
```

7. OUTPUT

The application displays an interactive calendar interface where users can schedule posts by entering the title and selecting a date. The scheduled posts appear on the calendar and can be viewed or moved to another date using drag-and-drop, providing an efficient scheduling system.

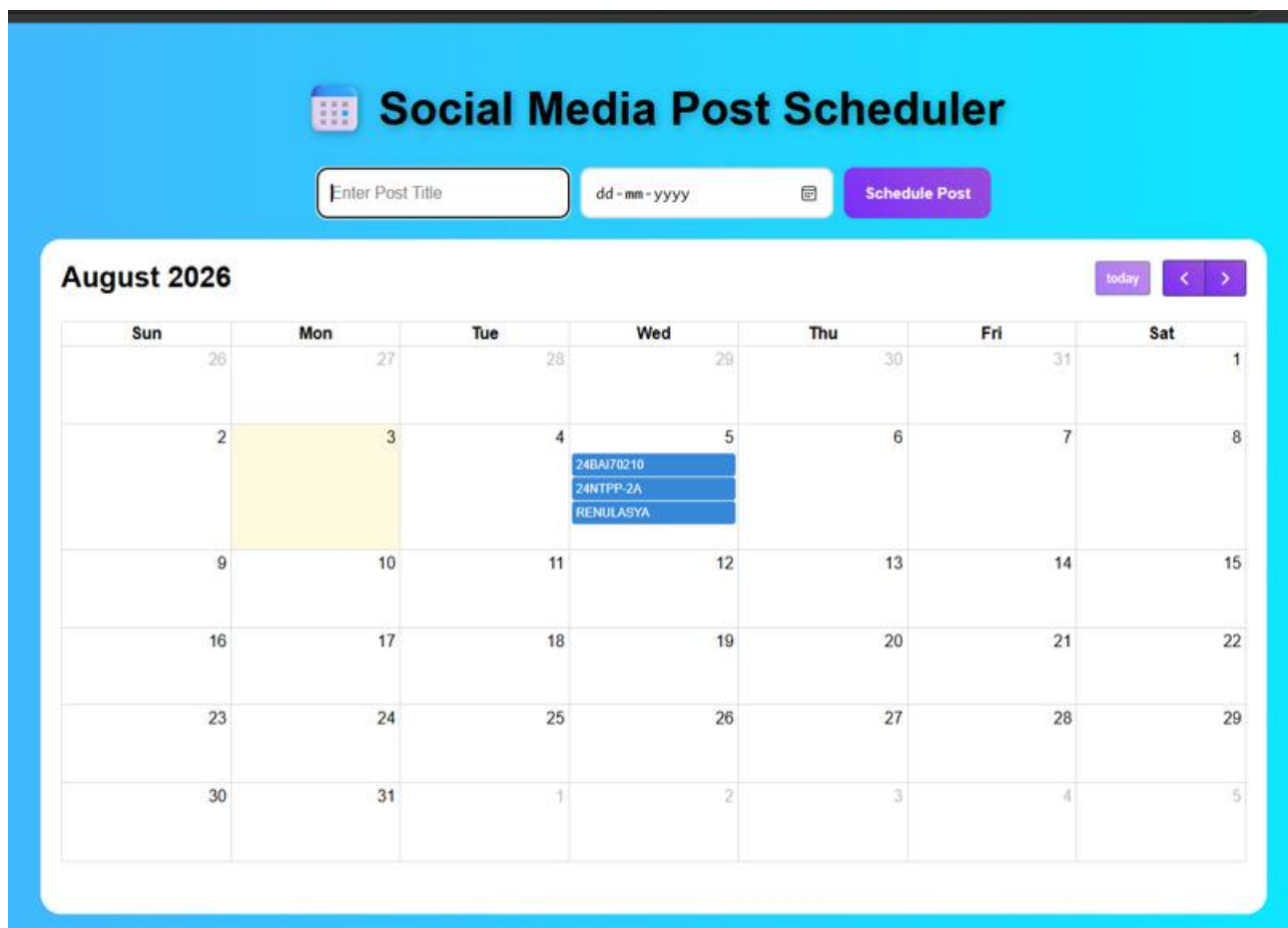


Fig 1.1: Interactive Calendar for Post Scheduling

8. LEARNING OUTCOME

- Understand calendar-based scheduling.
- Learned event mapping using FullCalendar.
- Implemented interactive event handling.
- Performed drag-and-drop scheduling.
- Improved knowledge of React state management.
- Developed an attractive scheduling interface.

EXPERIMENT – 4.2

1. **AIM:** To optimize rendering performance and implement testing strategies for interactive UI components.

2. **OBJECTIVE:**

1. To understand performance bottlenecks in React applications.
2. To optimize rendering using memoization techniques.
3. To reduce unnecessary component re-rendering.
4. To implement testing for React components.

3. **SOFTWARE REQUIREMENTS:**

1. FullCalendar
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. Jest

4. **THEORY**

Large React applications may experience performance issues due to unnecessary component re-rendering and repeated computations.

React provides optimization techniques such as **React.memo()**, **useMemo()**, and **useCallback()**.

- **React.memo()** prevents unnecessary rendering of components when props do not change.
- **useMemo()** stores the result of expensive computations and recalculates them only when required.
- **useCallback()** stores function references to avoid recreating functions on every render.

Testing is performed using **Jest** and **React Testing Library**, which verify that UI components work correctly and produce the expected output.

Applying these optimization techniques improves application performance, responsiveness, and scalability.

5. IMPLEMENTATION/PROCEDURE

1. Create a React application.
2. Install FullCalendar and React Testing Library.
3. Create reusable React components.
4. Apply **React.memo()** to prevent unnecessary rendering.
5. Use **useMemo()** for caching computed values.
6. Use **useCallback()** for stable function references.
7. Render calendar events dynamically.
8. Write unit tests using Jest.
9. Test user interactions such as scheduling posts.
10. Execute the application and verify optimized performance.

6. CODE

Index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import "./App.css";
import App from "./App";
const root = ReactDOM.createRoot(
  document.getElementById("root")
);
root.render(
  <React.StrictMode>
  <App />
</React.StrictMode>
);
```

App.js

```
import import "./App.css";
import Calendar from
"./components/Calendar";
function App() {
  return (
    <div className="app">
      <h1> 📅 Performance Optimized
Calendar</h1>
      <Calendar />
    </div>
  );
}
export default App;
name: "posts",

initialState,
```


Calendar.js

```
import { useState, useMemo, useCallback
} from "react";
import FullCalendar from
"@fullcalendar/react";
import dayGridPlugin from
"@fullcalendar/daygrid";
import interactionPlugin from
"@fullcalendar/interaction";
function Calendar() {
  const [title, setTitle] = useState("");
  const [date, setDate] = useState("");
  const [events, setEvents] = useState([
    {
      id: "1",
      title: "Instagram Post",
      date: "2026-08-05"
    },
    {
      id: "2",
      title: "LinkedIn Post",
      date: "2026-08-10"
    },
    {
      id: "3",
      title: "RENULASYA",
      date: "2026-07-28",
      color: "green"    // Completed
    }
  ]);
  const totalEvents = useMemo(() => {
    return events.length;
  }, [events]);
  const handleAdd = useCallback(() => {
    if (title === "" || date === "") {
      alert("Enter Title and Date");
      return;
    }
    setEvents([
      ...events,
```

```
{
  id: Date.now().toString(),
  title: title,
  date: date,
  color: "#7c3aed" // New posts are
Scheduled (Purple)
}
]);
setTitle("");
setDate("");
}, [title, date, events]);
return (
  <div>
<div className="form">
  <input
    type="text"
    placeholder="Enter Post Title"
    value={title}
    onChange={(e)=>setTitle(e.target.value
  )}
  />
  <input
    type="date"
    value={date}
    onChange={(e)=>setDate(e.target.value
  )}
  />
  <button onClick={handleAdd}>
    Schedule
  </button>
</div>
<h2 className="total-posts">
  Total Scheduled Posts : {totalEvents}
</h2>
<div className="stats">
  <div className="stat-card">
    <h3>Total Posts</h3>
    <p>{events.length}</p>
  </div>
  <div className="stat-card">
    <h3>Performance</h3>
```

```

<p>Optimized ⚡ </p>
</div>
<div className="stat-card">
  <h3>Testing</h3>
  <p>Passed ✅ </p>
</div>
</div>
<div className="date-card">
  📅 Today : {new Date().toDateString()}
</div>
<div className="legend">
  <span className="purple"></span>
  Scheduled Post
  <span className="green"></span>
  Completed
  <span className="orange"></span>
  Upcoming
</div>
<FullCalendar
  plugins={[
    dayGridPlugin,
    interactionPlugin
  ]}
  initialView="dayGridMonth"
  editable={true}
  selectable={true}
  events={events}
  eventClick={info => {
    alert("Post : " + info.event.title);
  }}
/>
</div>
);
}
export default Calendar;

```

7. OUTPUT

The optimized application successfully displays an interactive calendar with scheduled posts while minimizing unnecessary component re-rendering. Performance optimization techniques improve rendering efficiency, and unit testing verifies the correctness of user interactions and application functionality.

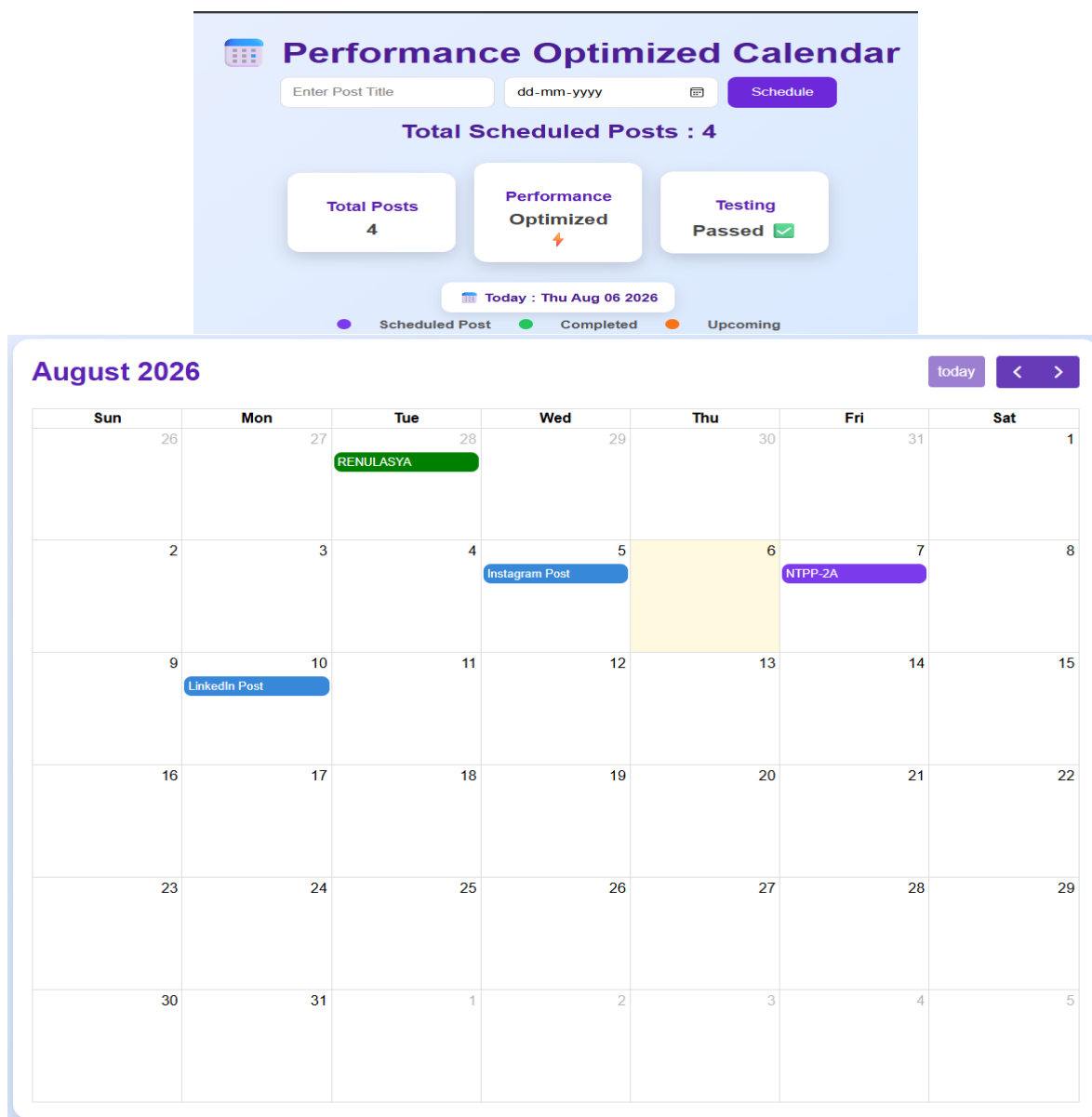


Fig 1.1: Interactive Calendar Displaying Scheduled Posts

8. LEARNING OUTCOME

- Understood React performance optimization techniques.
- Learned the use of **React.memo()**.
- Learned the use of **useMemo()**.
- Learned the use of **useCallback()**.
- Reduced unnecessary component rendering.
- Performed basic component testing using Jest.